

1. Objetivo do Projeto

O Pongtron 5001 é uma implementação completa do clássico jogo de arcade Pong realizada exclusivamente em hardware digital, sem uso de microprocessadores ou software. O circuito foi projetado no simulador Logisim-Evolution v4.0.0 e tem como objetivos principais:

- Demonstrar o uso integrado de lógica combinacional e sequencial para controle de jogos em tempo real.
- Implementar uma Máquina de Estados Finitos (FSM) implícita distribuída que gerencia o fluxo do jogo (início, andamento, pontuação e pausa).
- Prover datapath completo para movimentação da bola, colisões e placar.
- Suportar dois modos de jogo: Jogador vs Jogador (PvP) e Jogador vs Computador (PvC), com múltiplos níveis de dificuldade.
- Gerar a imagem do jogo em tempo real em uma matriz de pontos (Dot Matrix).
- Manter e exibir a pontuação de ambos os jogadores.

O projeto consolida conceitos de circuitos digitais como contadores, registradores, comparadores, multiplexadores, operações aritméticas (adição/subtração/multiplicação) e exibição em matrizes LED, todos conectados por barramentos de sinais (Tunnels)

3. Explicação da FSM

O Pongtron 5001 não possui uma FSM centralizada com registrador de estado único. Em vez disso, implementa uma FSM distribuída implícita cujos estados são definidos pela combinação de sinais de controle propagados via Tunnels:

Estado	Condição de Entrada	Comportamento
IDLE / NEW_GAME	IsNewGame=1 (reset ou power-on)	Bola posicionada no centro; placar zerado; direção sorteada.
RUNNING	IsRunning=1, IsPaused=0	Clock ativo, bola move, paletas respondem às entradas
PAUSED	IsPaused=1 (switch Pause)	LockUpdates=1; bola e paletas congeladas; PauseScreen exibida.
SCORING / RESET	IsOnLeftEdge=1 ou IsOnRightEdge=1	Incrementa placar do vencedor; bola retorna ao centro.
LOCK_MOVEMENT	LockMovement=1 (transição de ponto)	Paletas e bola travadas até o novo saque.

3.1 Sinais de Controle Globais

- **IsRunning:** habilita o clock para todos os contadores e flip-flops de posição.
- **IsPaused:** gerado pelo switch de pause; congela atualizações e exibe tela de pausa.
- **IsNewGame:** sinal de reset global; reinicia posições e zera placar.
- **LockMovement:** trava movimento da bola e paletas durante a transição de ponto.
- **LockScore:** impede contagem dupla durante o ciclo de reset após um ponto.
- **LockUpdates:** combinação de IsPaused e LockMovement.

3.2 Transições de Estado

As transições são implementadas combinacionalmente: o sinal **IsOnRightEdge** (bola ultrapassa a margem direita) e **IsOnLeftEdge** (bola ultrapassa a margem esquerda) ativam simultaneamente o **ScoreHandler** do respectivo jogador e disparam o **ResetPosition** da bola. A sequência é:

- Bola atinge borda → **IsOnLeftEdge** ou **IsOnRightEdge** = 1
- **ScoreHandler** incrementa (**LockScore**=0); **LockMovement** ativado
- **BallHandler** recebe **Reset**=1 → bola retorna ao **PlayfieldCenter**
- **BallStartDirectionHandler** sorteia nova direção; **Reset**=0; **LockMovement**=0
- Jogo retoma no estado **RUNNING**

4. Explicação do Datapath

O datapath do Pongtron 5001 é composto por unidades aritméticas e de controle dedicadas para cada entidade do jogo. Os barramentos principais são:

4.1 Datapath da Bola — BallHandler

A bola tem posição representada em dois registradores de 8 bits (**BallX** e **BallY**), atualizados por contadores bidirecionais:

- Dois Contadores (um para X, um para Y): incrementam ou decrementam conforme a direção atual da bola.
- Dois T Flip-Flops: armazenam o bit de direção (positivo/negativo) de cada eixo. São toggled quando ocorre colisão ou o sinal **ForceFlipX** é ativado (rebote em paleta).
- Dois XOR Gates: combinam o flip-flop de direção com o bit de enable do contador para obter sentido correto.
- Dois Multiplexadores: selecionam entre a posição atual e o centro do campo quando **Reset**=1.
- Dois Subtratores: calculam a distância da bola às bordas para detecção de colisão.

- Dois Comparadores: verificam se a bola atingiu as bordas superior/inferior.

4.2 Datapath da Paleta — PaddleHandler

Cada paleta tem posição Y de 5 bits e tamanho configurável:

- Contador: mantém a posição Y da paleta; habilitado conforme direção do jogador.
- Adicionador / Subtrator: calculam as bordas superior e inferior da paleta ($\text{PaddleY} \pm \text{PaddleSize}/2$) para detecção de intervalo.
- Dois Comparadores: verificam se a posição Y da bola está dentro do intervalo da paleta → saída `IsInPaddleRange`.
- AND Gate: combina `IsInPaddleRange` com a posição X para gerar a colisão final.
- Multiplicador: calcula deslocamento do sprite da paleta para o display.

4.3 Datapath do Placar — ScoreHandler / BCDConverter

O placar usa um contador de 16 bits por jogador:

- Counter (16 bits): incrementado no evento de ponto.
- BCDConverter: converte o binário em 5 dígitos decimais (unidades, dezenas, centenas, ...) usando 5 instâncias do módulo `ToDigit` (divisão + resto).
- CharacterDisplayHandler: mapeia cada dígito BCD para a ROM de caracteres e posiciona no display

4.4 Barramentos Principais

Barramento	Largura	Descrição
BallX / BallY	8 bits	Posição absoluta da bola no campo.
PaddleAPositionY / PaddleBPositionY	5 bits	Posição vertical das paletas.
PlayerAScore / PlayerBScore	16 bits	Placar de cada jogador.
PlayerAControl / PlayerBContro	2 bits	Direção do movimento (Up/Down/None).
Difficulty	2 bits	Nível de dificuldade da IA (0–3).
PaddleSize	5 bits	Tamanho configurável da paleta.
BallStartDirection	2 bits	Direção inicial da bola no saque.
IsColliding / ForceFlipX	1 bits	Evento de colisão detectado.

5. Lógica de Colisões

A detecção de colisões é feita puramente em hardware combinacional, dividida em dois módulos: BallCollisionHandler (colisão com paletas e paredes) e a lógica interna do BallHandler (rebote nas bordas superior/inferior).

5.1 Colisão Bola × Paredes Laterais (BallCollisionHandler)

O módulo recebe a posição X da bola e as posições X fixas dos dois jogadores (que definem a posição das paletas no eixo horizontal). Quatro Comparadores verificam:

- $BallPositionX \leq PlayerAPositionX$: bola na linha da paleta A (esquerda). Se $CanCollideWithPlayerA=1$ (habilitador de colisão por lado), $IsColliding=1$ e $IsOnLeftEdge$ é avaliado.
- $BallPositionX \geq PlayerBPositionX$: bola na linha da paleta B (direita). Se $CanCollideWithPlayerB=1$, $IsColliding=1$ e $IsOnRightEdge$ é avaliado.
- $IsOnLeftEdge$: bola passou da paleta A sem rebote → ponto para B.
- $IsOnRightEdge$: bola passou da paleta B sem rebote → ponto para A.

5.2 Colisão Bola × Paleta (PaddleHandler → BallHandler)

A verificação se a bola está na faixa vertical da paleta é feita no PaddleHandler:

- Calcula $PaddleTop = PaddleY - PaddleSize/2$ e $PaddleBottom = PaddleY + PaddleSize/2$ usando subtrator e adicionador.
- Dois comparadores: $BallY \geq PaddleTop$ e $BallY \leq PaddleBottom$.
- AND dos dois resultados → $IsInPaddleRange$.
- No BallCollisionHandler, a saída $IsColliding$ combina (AND) $IsInPaddleRange$ com a detecção de posição X → se ambos verdadeiros, $ForceFlipX=1$ é enviado ao BallHandler.

5.3 Rebote nas Bordas Superior/Inferior (BallHandler)

Dentro do BallHandler, dois comparadores verificam se BallY atingiu 0 (topo) ou o limite máximo do campo (base). Quando verdadeiro, o T Flip-Flop do eixo Y recebe toggle, invertendo a direção vertical. O eixo X é invertido via ForceFlipX proveniente do BallCollisionHandler.

5.4 Habilitadores de Colisão (CanCollide)

Para evitar colisões repetidas (bola 'presa' na paleta), os sinais CanCollideWithPlayerA e CanCollideWithPlayerB são controlados: após uma colisão com a paleta A, CanCollideWithPlayerA é desabilitado até que a bola se afaste suficientemente, prevenindo múltiplos flips consecutivos.

6. Estratégia da Paleta Automática (IA)

O módulo `ComputerEnemyMovementHandler` implementa a lógica de movimentação automática da paleta controlada pelo computador. A estratégia é baseada em seguir a bola com restrição de velocidade controlada pelo nível de dificuldade:

6.1 Algoritmo de Seguimento

- Entrada: `BallY` (8 bits) e `PaddlePositionY` (8 bits).
- Um Comparador verifica se `BallY > PaddleY` (bola acima do centro da paleta): se sim, a paleta deve subir; caso contrário, descer.
- Um Multiplexador seleciona o sinal de movimento (`Up=01` / `Down=10` / `None=00`) com base nessa comparação.
- O resultado é a saída `Movement` (2 bits) enviada ao `PlayerHandler`.

6.2 Controle de Velocidade por Dificuldade

A velocidade da IA é regulada por um Counter divisor de clock e o sinal `Difficulty` (2 bits, 0–3):

- `Difficulty = 0`: IA muito lenta — move apenas 1 em cada 8 ciclos.
- `Difficulty = 1`: IA lenta — 1 em cada 4 ciclos.
- `Difficulty = 2`: IA média — 1 em cada 2 ciclos.
- `Difficulty = 3`: IA rápida — move a cada ciclo de clock.

Um NOT Gate e Ground/Power completam a lógica de enable do contador de velocidade. O sinal `LockMovement` anula o movimento durante transições.

6.3 Limitações da Estratégia

- A IA segue a posição Y da bola diretamente, sem prever trajetória — facilmente derrotada em dificuldades baixas.
- Não há lógica de antecipação ou rebote calculado.
- A limitação de velocidade é o único parâmetro de balanceamento.

8. Exemplos de Funcionamento

8.1 Início de Partida

- Ao ligar o circuito (`Reset=1` liberado), `IsNewGame=1` é ativado.
- `BallHandler` posiciona a bola em (`PlayfieldCenterX=32`, `PlayfieldCenterY=16`).
- `BallStartDirectionHandler` define a direção inicial: se `ScoreA > ScoreB`, bola vai para o lado de B; caso contrário, para o lado de A.
- `PaddleHandlers` resetam as paletas para o centro vertical.
- `IsNewGame=0`, `IsRunning=1` → jogo em andamento.

8.2 Rebote na Paleta

- BallX atinge PlayerAPositionX (coluna da paleta A, e.g. X=4).
- BallCollisionHandler: CanCollideWithPlayerA=1 → verifica IsInPaddleRange.
- PaddleHandler A: BallY=18, PaddleY=16, PaddleSize=8 → PaddleTop=12, PaddleBottom=20 → IsInPaddleRange=1.
- IsColliding=1 → ForceFlipX=1 enviado ao BallHandler.
- T Flip-Flop do eixo X sofre toggle: direção inverte (bola passa a se mover para direita).
- CanCollideWithPlayerA=0 temporariamente (evita duplo rebote).

8.3 Ponto Marcado

- Bola ultrapassa PlayerBPositionX sem colisão → IsOnRightEdge=1.
- ScoreHandler A: LockScore=0, Increment=1 → Score A incrementa.
- LockMovement=1: bola e paletas congeladas.
- BallHandler: Reset=1 → bola retorna ao centro.
- Após 1 ciclo: Reset=0, LockMovement=0 → novo saque.

8.4 Modo Computador — Dificuldade 2

- PlayerB configurado com IsComputerPlayer=1, Difficulty=10 (binário = 2).
- ComputerEnemyMovementHandler: clock dividido por 2 → paleta B move a cada 2 ciclos.
- BallY=24, PaddleB_Y=14 → comparador: BallY > PaddleY → Movement=Down (10b).
- PlayerHandler B repassa Movement para PaddleHandler B, que decrementa PaddleY.
- Com dificuldade 0, a IA perde facilmente contra jogadores experientes.

8.5 Pause

- Jogador ativa Switch Pause → IsPaused=1.
- LockUpdates=1: todos os contadores de posição são desabilitados.
- DisplayDriver ativa PauseScreenHandler, exibindo 'PAUSE' no DotMatrix.
- LED de pausa acende.
- Ao desativar o switch, jogo retoma do estado anterior sem perda de posição.

9. Conclusão e Possíveis Melhorias

O Pongtron 5001 demonstra com sucesso a viabilidade de implementar um jogo completo e interativo utilizando exclusivamente hardware digital combinacional e sequencial. A arquitetura modular facilita a compreensão de cada subsistema e a hierarquia de circuitos no Logisim-Evolution torna o projeto escalável.

9.1 Conclusões Técnicas

- A FSM distribuída via sinais de controle (LockMovement, IsPaused, IsNewGame) é uma alternativa eficiente a um registrador de estado centralizado para jogos simples.
- O uso de T Flip-Flops para armazenar a direção da bola é elegante e minimiza a lógica necessária para o rebote.
- A ROM de sprites para paletas e caracteres separa claramente dados de lógica, seguindo boas práticas de design digital.
- O DisplayDriver com OR em cascata é funcional, mas o número de gates cresce linearmente com os layers — aceitável para este escopo.
- A IA baseada em seguimento direto é simples e eficaz para o propósito educacional.

9.2 Possíveis Melhorias

Melhoria	Descrição e Implementação
IA com predição de trajetória	Calcular onde a bola estará quando atingir a linha da paleta, em vez de seguir a posição Y atual. Requer um multiplicador de velocidade e somador de posição.
Angulo de rebote variável	Fazer o ângulo de saída da bola depender do ponto de impacto na paleta (centro → reto; extremidades → Calcular onde a bola estará quando atingir a linha da paleta, em vez de seguir a posição Y atual. Requer um multiplicador de velocidade e somador de posição.
Aceleração progressiva	Aumentar a velocidade da bola a cada rebote na paleta, tornando a partida mais desafiadora. Implementável com um contador de rebotes e um divisor de clock variável.
Placar persistente	Usar RAM ou registradores adicionais para manter o placar entre partidas.
Efeitos sonoros	Adicionar um buzzer piezoelétrico controlado por eventos de colisão e ponto (módulo de áudio simples com frequência programável).
Modo torneio / melhor de N	Adicionar lógica de 'set' com contador de rodadas e exibição do vencedor.
Otimização do DisplayDriver	Substituir os OR gates em cascata por um multiplexadr de prioridade, reduzindo profundidade lógica e melhorando timing.
Síntese para FPGA	Exportar o design para VHDL/Verilog e sintetizar em uma FPGA real (ex: Basys 3)

	com VGA output para display em monitor.
--	---